

# A Search Reduction from Set Cover to Exact DNF

Working manuscript for expert review  
2026-09-24

## Abstract

We give deterministic polynomial-time construction and recovery maps from Set Cover to DNF minimization when the target function is specified by an explicit list of all accepted assignments. Unlisted assignments are rejected. A SAT encoding of the source is transformed into a three-element exact-cover instance. One-hot downsets and a parity completion then produce the total Boolean function. From every DNF within the term bound, the recovery map extracts an exact cover, a satisfying SAT assignment, and a source cover. It also handles the target’s no-solution output. The downset and parity steps reconstruct known DNF-hardness machinery; the contribution here is an executable composition and a decoder for the fixed search contract. Finite Z3 and Kissat checks support the implementation separately from the proof.

## 1. Introduction

DNF minimization can be viewed as covering accepted assignments by implicant cubes. Its input representation matters: an arbitrary partial truth table may leave exponentially many assignments unconstrained, while an explicit accepted list defines a total function. Allender, Hellerstein, McCabe, Pitassi, and Saks [1, Lemmas 3.1 and 3.3] describe a set-cover downset construction and a parity completion. They note that a one-hot construction has a polynomial accepted list when each source set has bounded size. This paper instantiates that route for an unrestricted Set Cover search input and specifies recovery from every valid target answer.

**Theorem 1 (search reduction).** For the encodings in Section 2, deterministic algorithms  $F$  and  $G$  run in polynomial time and satisfy

$$\forall x \in I_{\text{SC}}, \forall y \in S_{\text{DNF}(F(x))}, G(x, y) \in S_{\text{SC}(x)}. \quad (1)$$

The construction first expresses the source cover condition as CNF with a cardinality circuit. A three-element exact-cover gadget converts that CNF into a bounded-set covering instance. The final accepted list uses the downsets of those triples and two parity variables. The gadget appears in Figure 1; the exact accepted-list construction and output recovery are proved in Section 3.4 and Section 4. Theorem 1 is a reconstruction of known DNF-hardness machinery for a specific two-map search contract; it makes no new hardness or approximation claim.

## 2. Problems and output semantics

A Set Cover input has universe  $U = \{0, \dots, u-1\}$ , explicitly listed subsets  $S_1, \dots, S_m \subseteq U$ , and a nonnegative bound  $k$ . A valid output is a list of at most  $k$  distinct set indices whose union is  $U$ . The string NO-SOLUTION is valid exactly when no such list exists. Duplicate and empty input sets are allowed. Renumbering an arbitrary finite universe gives this representation without changing its covering relation.

The target input has  $n$  Boolean variables, an explicit set  $A \subseteq \{0, 1\}^n$  of accepted assignments, and a nonnegative term bound  $K$ . A target witness is a DNF with at most  $K$  conjunctions of literals whose satisfying set is exactly  $A$ . Each term is encoded by a vector in  $\{-1, 0, 1\}^n$ : a negative literal, an absent variable, or a positive literal. The same NO-SOLUTION string is valid exactly when no such DNF exists. In particular, every assignment outside  $A$  must be rejected. The encodings include  $n = 0$ , whose sole assignment is the empty string.

The map  $F$  reads the source input and emits a target instance. The map  $G$  reads the source input and a valid target output; it reconstructs all metadata from that source input. Neither map invokes an optimization or satisfiability solver.

### 3. Construction

#### 3.1. Polynomially decidable branches

If  $U$  is empty, the empty cover is known. If an element belongs to no source set, or if  $k = 0$  and  $U$  is nonempty, infeasibility is known. The implementation also checks every singleton or pair when  $k \leq 2$ , and selects all sets when  $k \geq m$  and their union is  $U$ . These checks take polynomial time. A known YES case maps to  $n = 0$ ,  $A = \{\varepsilon\}$ ,  $K = 1$ ; a known NO case maps to  $n = 1$ ,  $A = \{0\}$ ,  $K = 0$ . Recovery recomputes the corresponding source answer. For the remaining construction, assume  $3 \leq k < m$  and every universe element occurs in a source set.

#### 3.2. Cover condition as CNF

Let  $x_j$  indicate selection of  $S_j$ . For each  $e \in U$ , add the nonempty clause  $\bigvee_{j:e \in S_j} x_j$ . A Tseitin circuit enforces the bound. Let  $s_{i,j}$  mean that at least  $j$  of  $x_1, \dots, x_i$  are true, with  $s_{i,0} = 1$  and  $s_{0,j} = 0$  for  $j > 0$ . For  $1 \leq j \leq k+1$  its recurrence is

$$s_{i,j} = s_{i-1,j} \vee (x_i \wedge s_{i-1,j-1}). \quad (2)$$

Every AND and OR gate receives equivalence clauses, and a final unit clause requires  $\neg s_{m,k+1}$ . A satisfying assignment therefore chooses at most  $k$  sets and covers every element. Conversely, every source cover extends through the gates to a satisfying assignment. We call the resulting CNF  $\Phi$ .

#### 3.3. A three-element exact-cover gadget

Index each literal occurrence  $o$  of  $\Phi$  separately. Introduce four points  $a_o, b_o, T_o, F_o$ . If a variable has occurrences  $o_1, \dots, o_r$  in cyclic order, add two triples for each  $i$ :

$$H_i = \{a_i, b_i, F_i\}, \quad L_i = \{a_i, b_{i+1}, T_i\}, \quad b_{r+1} = b_1. \quad (3)$$

The two  $H$  triples shown in Figure 1 choose the true mode and leave the  $T$  tips free. The  $L$  triples choose the false mode and leave the  $F$  tips free. Every exact cover selects one wheel triple per  $a_i$ . The  $b_i$  points force all wheel choices for a variable to use the same mode: a mode change would leave one  $b_i$  uncovered and another covered twice.

| triple                                      | points |       |       |       |       |       |       |       |       |       |
|---------------------------------------------|--------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
|                                             | $a_1$  | $a_2$ | $b_1$ | $b_2$ | $T_1$ | $T_2$ | $F_1$ | $F_2$ | $c_A$ | $c_B$ |
| $H_1$                                       | ●      | ○     | ●     | ○     | ○     | ○     | ●     | ○     | ○     | ○     |
| $H_2$                                       | ○      | ●     | ○     | ●     | ○     | ○     | ○     | ●     | ○     | ○     |
| $L_1$                                       | ●      | ○     | ○     | ●     | ●     | ○     | ○     | ○     | ○     | ○     |
| $L_2$                                       | ○      | ●     | ●     | ○     | ○     | ●     | ○     | ○     | ○     | ○     |
| <hr style="border-top: 1px dashed black;"/> |        |       |       |       |       |       |       |       |       |       |
| $C_1$                                       | ○      | ○     | ○     | ○     | ●     | ○     | ○     | ○     | ●     | ●     |

H: true wheel L: false wheel C: positive clause literal at occurrence 1

Figure 1: Exact incidence matrix for a variable with two literal occurrences. A filled circle means that the row's three-element set contains the column's point. Rows  $H_1, H_2$  are the true wheel choice;  $L_1, L_2$  are the false choice. The row  $C_1 = \{c_A, c_B, T_1\}$  attaches a positive unit clause to occurrence 1. The dashed line separates wheel and clause sets.

For a clause with  $\ell \geq 1$  literal occurrences, add clause points  $c_A, c_B$ . For each positive occurrence  $o$ , add  $\{c_A, c_B, T_o\}$ ; for each negative occurrence, add  $\{c_A, c_B, F_o\}$ . Add  $\ell - 1$  pairs of garbage points. Each pair can be covered with either tip of any occurrence in that clause. An exact cover uses exactly one clause triple, whose tip is free precisely when that literal is true. Its garbage triples consume the other  $\ell - 1$  free tips. A satisfying assignment selects wheel modes, one true literal per clause, and the garbage triples; these choices form an exact cover.

Let  $R$  be the number of literal occurrences and  $C$  the number of clauses. The points split into three parts of size  $2R$  each: occurrence points contribute  $R, R, 2R$  and clause and garbage pairs contribute  $R, R, 0$ . Every triple uses one point from each part. The resulting universe has  $N = 6R$  points. A cover by at most  $2R$  triples is necessarily exact. The gadget therefore has such a cover if and only if  $\Phi$  is satisfiable. From every exact cover, the wheel modes recover a satisfying assignment.

### 3.4. Explicit accepted assignments

Let  $\mathcal{T}$  be the family of gadget triples. Represent its  $N$  points by one-hot vectors  $v_i \in \{0, 1\}^N$ . For each  $S \in \mathcal{T}$ , let  $w_S$  be its incidence vector, and let  $D$  be the union of all Boolean downsets below the  $w_S$ . Set  $V = \{v_i : i = 1, \dots, N\}$  and  $R_* = D - V$ , with  $s = |R_*|$ . Because each  $w_S$  has exactly three ones, each downset has eight vectors. So  $s \leq 1 + 4|\mathcal{T}|$ : it contains the zero vector and at most the three pairs and one triple contributed by each set.

For  $z \in \{0, 1\}^N$ , let  $p(z)$  be its parity. The target has  $n = N + 2$  variables and the following exact accepted set and term bound:

$$A = \{(z, 1, 1) : z \in D\} \cup \{(z, p(z), 1 - p(z)) : z \in R_*\}, \quad K = s + 2R. \quad (4)$$

The algorithm enumerates the zero, pair, and triple masks of each member of  $\mathcal{T}$ , together with all one-hot masks. It writes only the assignments in Equation 4; it never scans the full Boolean cube.

## 4. Recovery and correctness

**Lemma 1 (forward witnesses).** If the source has a cover of at most  $k$  sets, the target in Equation 4 has a DNF of at most  $K$  terms.

**Proof.** The source cover extends to a satisfying assignment of  $\Phi$  by Equation 2, so it gives a cover of the gadget by  $q \leq 2R$  triples. For each selected triple  $S$ , take the partial-function term that fixes every zero coordinate of  $w_S$  negatively. Its satisfying vectors lie in the downset of  $w_S$ , and these terms cover all one-hot vectors. Append two positive auxiliary literals to each such term. For every star  $z \in R_*$ , also take a term fixing all  $N$  coordinates to  $z$  and fixing the auxiliary coordinate that is one in  $(p(z), 1 - p(z))$ . This star term covers both  $(z, 1, 1)$  and its parity point. The resulting DNF accepts exactly Equation 4 and uses  $s + q \leq K$  terms.  $\square$

**Lemma 2 (all valid target DNFs).** Every exact DNF for Equation 4 with at most  $K$  terms yields a source cover of size at most  $k$  in polynomial time.

**Proof.** Consider a parity point  $(z, p(z), 1 - p(z))$  for  $z \in R_*$ . Any term covering it must fix every original coordinate. Otherwise flipping a free original bit reverses parity while preserving the auxiliary bits, producing a rejected assignment. Distinct parity points therefore need distinct terms. Such terms cannot cover a one-hot point because  $R_*$  excludes one-hot vectors.

At least  $s$  terms are reserved for the parity points. Any term covering  $(v_i, 1, 1)$  must include both positive auxiliary literals, since changing either bit gives a rejected point. Remove those literals and let  $u$  be the maximal original vector satisfying the remaining term. Exactness implies  $u \in D$ , so  $u \leq w_S$  for some gadget triple  $S$ . Assign one such triple to the term. Every one-hot point covered by that term names an element of  $S$ . At most  $K - s = 2R$  assigned triples cover all  $N = 6R$  gadget points. They form an exact cover, so the wheel modes yield a satisfying assignment of  $\Phi$ . Its first  $m$  variable values give a source cover of at most  $k$  sets. This decoder applies to nonprime, duplicate, redundant, and alternate valid DNFs.  $\square$

**Proof of Theorem 1.** The easy branches have direct correct outputs. On the hard branch, Lemma 1 maps every source cover to a valid target witness. Lemma 2 maps every valid target DNF to a source cover. If the target answer is NO-SOLUTION, Lemma 1 rules out a source cover, so  $G$  returns the valid source NO-SOLUTION answer. These cases exhaust  $S_{\text{DNF}(F(x))}$ , proving Equation 1.  $\square$

## 5. Polynomial bounds

Let  $L$  denote the source encoding length. The hard branch has  $u, m, k = O(L)$ :  $k < m$ , and every universe element must occur in the explicit incidence lists. The cardinality circuit has  $O(mk)$  gates. Its CNF has  $R = O(L + mk)$  literal occurrences and clauses of length at most  $\max(m, 3)$ . The gadget has  $N = 6R$  points and  $T = O(Rm)$  triples, including the local garbage choices. The accepted list contains at most  $N + 2(1 + 4T)$  strings of length  $N + 2$ . Construction and output length are therefore polynomial; a coarse bound is  $O((N + T)N)$  plus sorting and JSON overhead. The binary representation of  $K$  uses  $O(\log(N + T))$  bits.

Recovery rebuilds the CNF and gadget from  $x$ , scans the given target terms, maps each maximal original vector to a containing triple, and reads one wheel marker per source set variable. These operations and the output length are polynomial in  $L + |y|$ . Neither map depends on process memory from the other invocation or on a solver.

## 6. Discussion

Theorem 1 supplies an explicit-list search reduction with recovery from every valid output. The principal cost is the gadget expansion followed by long accepted strings; on a small tested source, the target already reached 1,838 variables and 14,870 accepted assignments. The result is a reproducible reconstruction of known DNF-hardness ingredients. It does not improve their complexity classification or show practical minimization efficiency. The finite checks in Section A test implementation behavior and leave the universal claim to the proof.

## References

[1] Eric Allender, Lisa Hellerstein, Paul McCabe, Toniann Pitassi, and Michael E. Saks, “Minimizing Disjunctive Normal Form Formulas and  $\text{AC}^0$  Circuits Given a Truth Table,” *SIAM Journal on Computing* 38(1) (2008), 63–84. DOI: 10.1137/060664537. The downset and parity steps used here are Lemmas 3.1 and 3.3; §3 discusses the explicit accepted-list variant.

## A. Verification and reproducibility

The repository fixes 115 distinct source cases before candidate construction: 100 seeded random cases and 15 designed edges, including 54 YES and 61 NO. The prepared Z3 checker recomputed source answers exhaustively, validated witnesses, and solved 133 actual target outputs for the final candidate. Sixty-one target NO-SOLUTION outputs passed through recovery. Nine inputs used the full gadget; the rest used the direct polynomial branches. A separate Kissat verifier checked four hand-selected inputs and five outputs, including two DNFs for a hard YES target and a conclusive hard NO target. These are finite checks, not a proof of Theorem 1.

The run used Python 3.12.14, uv 0.12.17, Z3 5.1.0 (z3-solver 5.1.0.0 in the locked project), Kissat 4.0.4, and Typst 0.15.1. The capability probe also found Lean 4.34.0 and Lake 5.0.0; formalization was not requested and Mathlib was pending. CP-SAT was not used. From the repository root, rebuild the Python environment and rerun the checks with:

```
cd campaigns/set-cover-minimum-dnf/work
uv sync --locked
uv run --locked python3 check.py --self-test
uv run --locked python3 check.py --candidate algorithm.py
uv run --locked python3 verify.py --candidate algorithm.py
```

The first check prints the corpus and 115-case self-test result. The candidate check prints 115 source cases, 133 target outputs, 61 NO-SOLUTION outputs. The separate verifier prints 4 source cases, 5 target outputs. It requires the Kissat executable on PATH. Its exact-cover shortcut applies to the tight residual instances in that finite test set; it raises an error elsewhere. The prepared checker has no finite variable-count cutoff, but sparse-cube enumeration and exact solving can become expensive.

Both map commands use JSON on standard input and output. For example, from the work/ directory:

```
cat <<'JSON' | uv run --locked python3 algorithm.py
{"universe_size":0,"sets":[],"k":0}
JSON
cat <<'JSON' | uv run --locked python3 algorithm.py --extract
{"source":{"universe_size":0,"sets":[],"k":0},
 "target_solution":{"terms":[[]]}}
JSON
```

These commands emit the legal target {"n":0,"accepted":[""],"K":1} and the source witness {"cover":[]}. The exact corpus, generator, dependency lock, candidate, checker and retained output logs are in the repository. Compile this paper from campaigns/set-cover-minimum-dnf/work/ with typst compile manuscript.typ manuscript.pdf.